FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

BACHELOR THESIS

Jan Burda

Optimizing 3D Bin Packing Using Evolutionary Algorithms

Department of Theoretical Computer Science and Mathematical Logic

Supervisor of the bachelor thesis: RNDr. Jiří Fink, Ph.D.
Study programme: Informatika

Prague 2026

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date ....

Author's signature

Dedication.

Title: Optimizing 3D Bin Packing Using Evolutionary Algorithms

Author: Jan Burda

Department: Department of Theoretical Computer Science and Mathematical Logic

Supervisor: RNDr. Jiří Fink, Ph.D., Department of Theoretical Computer Science and Mathematical Logic

Abstract: Abstract

Keywords: 3D Bin Packing, Evolutionary Algorithms

Contents

Introduction **6**
0.1 Globalization and Containers . . . 6
0.2 Container Packing Problems in Computer Science . . . 6
0.2.1 3D Bin Packing Problem . . . 6
0.2.2 Knapsack Loading Problem . . . 6
0.2.3 Container Loading Problem . . . 7
0.3 State of art algorithms . . . 7
0.4 Contribution . . . 7

1 Terminology and Problem Description **8**
1.1 Coordinates . . . 8
1.2 Sizes . . . 8
1.3 Region . . . 8
1.4 Rotation . . . 9
1.5 Relationships Between *Regions* . . . 9
1.6 Problem Description . . . 10
1.7 Fitness Function . . . 11
1.8 Data Representation . . . 12
1.8.1 Input . . . 12
1.8.2 Output . . . 12
1.8.3 Datasets . . . 12

2 The Packing Algorithm **13**
2.1 Terminology . . . 13
2.1.1 Pending box . . . 13
2.1.2 Placement Heuristic . . . 14
2.1.3 Opened Container . . . 14
2.1.4 Placed Box . . . 14
2.1.5 Packing Rules . . . 14
2.1.6 Packing Rules Fitness . . . 15
2.2 Empty Maximal Regions . . . 15
2.2.1 Update of a Single Empty Maximal Region . . . 16
2.2.2 Update of an Empty Maximal Regions List . . . 17
2.3 Heuristics . . . 18
2.3.1 Packing Order Heuristics . . . 18
2.3.2 Placement Heuristics . . . 18
2.4 The Box Packing Algorithm . . . 18

3 Packing Rules Evolution **20**
3.1 Hill Climbing . . . 20
3.2 Simulated Annealing . . . 20
3.3 Elitist Genetic Algorithm . . . 21
3.4 Memetic Extension of the Elitist Genetic Algorithm . . . 22

Bibliography **23**

Introduction

0.1 Globalization and Containers

The nature of today's world is greatly influenced by globalization, which not only connects economies and societies across continents, but also has a profound impact on cultures [1].

One of the main drivers of globalization is the transportation of goods. In order to transport goods effectively, containers are essential, as they allow for the easy movement of goods using various methods of transportation. They are used in trucks, ships, trains, and even airplanes, facilitating a smooth transition of goods when changing the mode of transport. The use of containers has tremendously reduced handling times and costs, making the global supply chain much more efficient [2].

Since the cost of shipping is typically calculated based on the number of containers, their volume, and their weight, any unused space consequently results in increased expenses. In large-scale logistics, where massive quantities of containers are shipped daily, even small improvements in packing efficiency can lead to significant cost savings as well as reduction in emissions. For example, in 2018, containers imported into the United States were, on average, loaded to only about 65% of their capacity [3].

0.2 Container Packing Problems in Computer Science

In the context of computer science and optimization, container packing problems generally revolve around three key types [4]:

0.2.1 3D Bin Packing Problem

The goal is to pack a set of rectangular boxes into the smallest possible number of containers, considering all three dimensions of the objects and preventing them from colliding.

0.2.2 Knapsack Loading Problem

Each box has an associated value, and the objective is to select a subset of boxes that fits into a single container while maximizing the sum of their values.

0.2.3 Container Loading Problem

The aim is to pack all boxes into a single container, which has infinite length, while minimizing the total volume used.

All of these problems are indeed NP-hard [4], so heuristic and approximation algorithms are usually used to find good solutions in a reasonable amount of time.

0.3 State of art algorithms

*TODO

0.4 Contribution

*TODO

1 Terminology and Problem Description

In this chapter, we will focus on the formal description of the extended 3D Bin Packing Problem. To make this possible, we first need to introduce the terminology related to the representation of 3D space and the objects within it.

1.1 Coordinates

A *Coordinates* is defined as a triplet (x, y, z), representing a position relative to the origin $(0, 0, 0)$, where:

- x denotes the position along the horizontal axis,
- y denotes the position along the vertical axis,
- z denotes the position along the depth axis.

Each coordinate value must be a non-negative integer:

$$x, y, z \in \mathbb{Z}_{\geq 0}.$$

The origin $(0, 0, 0)$ serves as the reference point from which all coordinates are measured.

1.2 Sizes

A *Sizes* of a rectangular object is defined as a triplet (X, Y, Z), where:

- X denotes the length of the edge along the horizontal axis (X-axis),
- Y denotes the length of the edge along the vertical axis (Y-axis),
- Z denotes the length of the edge along the depth axis (Z-axis).

Each of these values must be a non-negative integer:

$$X, Y, Z \in \mathbb{Z}_{\geq 0}.$$

1.3 Region

A *region* is a rectangular object defined by a pair of coordinates $(\mathit{Start}, \mathit{End})$, where:

- *start* denotes the coordinates of the vertex that is closest to the origin $(0, 0, 0)$,
- *end* coordinates denotes the vertex that is furthest from the origin $(0, 0, 0)$.

It is required that the start coordinate is less than or equal to the end coordinate along each axis:

$$\mathit{Start}_x \leq \mathit{End}_x, \quad \text{and} \quad \mathit{Start}_y \leq \mathit{End}_y, \quad \text{and} \quad \mathit{Start}_z \leq \mathit{End}_z.$$

1.4 Rotation

A *Rotation* of an item is defined by permuting its dimensions (X, Y, Z) according to the orientation, represented by an element of the set:

$$\{XYZ, XZY, YXZ, YZX, ZXY, ZYX\}$$

where each orientation corresponds to a rearrangement of the dimensions:

$$\begin{aligned}
XYZ &\to (X, Y, Z) \\
XZY &\to (X, Z, Y) \\
YXZ &\to (Y, X, Z) \\
YZX &\to (Y, Z, X) \\
ZXY &\to (Z, X, Y) \\
ZYX &\to (Z, Y, X).
\end{aligned}$$

This approach maintains the values as non-negative integers but changes their assignment to axes, effectively rotating the item in discrete 90-degree steps.

1.5 Relationships Between *Regions*

Given two *regions* $A = (A_{\text{start}}, A_{\text{end}})$ and $B = (B_{\text{start}}, B_{\text{end}})$, we define several spatial relationships.

All comparisons between coordinates are interpreted componentwise, for instance, $A_{\text{start}} \leq B_{\text{start}}$ means:

$$A^x_{\text{start}} \leq B^x_{\text{start}} \quad \text{and} \quad A^y_{\text{start}} \leq B^y_{\text{start}} \quad \text{and} \quad A^z_{\text{start}} \leq B^z_{\text{start}}.$$

Subregion

We say that region A is a *subregion* of region B if:

$$A_{\text{start}} \geq B_{\text{start}} \quad \text{and} \quad A_{\text{end}} \leq B_{\text{end}},$$

meaning that A lies entirely within the boundaries of B in all three dimensions. We write $A \subseteq B$.

Overregion

We say that region A is an *overregion* of region B if B is a subregion of A:

$$A_{\text{start}} \leq B_{\text{start}} \quad \text{and} \quad A_{\text{end}} \geq B_{\text{end}}.$$

We write $A \supseteq B$.

Intersection

Two regions A and B *intersect* if there is a non-empty volume that they share. Formally, define open intervals for each dimension:

$$I_A^d = (A_{\text{start}}^d, A_{\text{end}}^d), \quad I_B^d = (B_{\text{start}}^d, B_{\text{end}}^d), \quad \text{for } d \in \{x, y, z\}.$$

The regions intersect if and only if the intersection of intervals is non-empty in all dimensions:

$$A \cap B \neq \emptyset \iff \forall d \in \{x, y, z\} : I_A^d \cap I_B^d \neq \emptyset.$$

This definition ensures that the overlapping region has strictly positive volume.

Volume Function

We define a *volume function* V that returns the volume of *Sizes* or *Regions*:

- given **dimensions** $D = (X, Y, Z)$, the volume is:

$$\mathrm{V}(D) = X \cdot Y \cdot Z,$$

- given a **region** $R = (R_{\text{start}}, R_{\text{end}})$, the volume is defined as:

$$\mathrm{V}(R) = (R_{\text{end}}^x - R_{\text{start}}^x) \cdot (R_{\text{end}}^y - R_{\text{start}}^y) \cdot (R_{\text{end}}^z - R_{\text{start}}^z).$$

The result of V is always a non-negative integer (possibly zero).

1.6 Problem Description

We are given a rectangular *input container* C_in with sizes S and a maximum allowed weight $W_{\max}$, and a set of k rectangular *input boxes*. We call this pair the *packing input*. Each input box $i = 1, \ldots, k$ has sizes s_i and weight w_i. We model the container as a *region* $R_c = (\mathbf{0}, D)$, where $\mathbf{0} = (0, 0, 0)$ denotes the origin, and R determines the *end* coordinate of the container region. Similarly, when a box i is placed inside the container with a valid rotation Rot, it occupies a region $r_i = (p_i, p_i + Rot(d_i))$, where p_i is the start coordinate of the input box within the container. We require that r_i is a subregion of R_c as an effect of constraint C2 below.

Let B_c denote the set of boxes placed in container c. We define the following:

- the total weight of container c is:

$$W_c = \sum_{i \in B_c} w_i,$$

- the extension of the volume function for total volume occupied in container c is:

$$V(c) = \sum_{i \in B_c} \mathrm{V}(r_i).$$

Constraints

We impose the following constraints:

(C1) each box i is placed in exactly one container,

(C2) each box i must lie entirely within its container c:

$$r_i \subseteq R_c,$$

(C3) no two boxes i and j within the same container c may intersect:

$$\forall i \neq j \in B_c : \quad s_i \cap s_j = \emptyset,$$

(C4) the total weight of any container c does not exceed its weight limit:

$$W_c \leq W_{\max}$$

,

(C5) boxes may only be rotated by 90° along any axis,

(C6) every coordinate of a box's base must coincide either with a coordinate of the container floor or with a coordinate on the top surface of another box.

Objective

The goal is to assign all boxes to containers in a way that satisfies all constraints (C1)–(C6), while minimizing the number of containers used N. Compared to the classical 3D bin packing problem, this version introduces two key additional constraints: the maximum weight limit (C4), and the full-support condition (C6) that ensures physical stability of stacked boxes.

1.7 Fitness Function

The objective of the fitness function is to numericaly represent the quality of the solution. The lower the fitness, the better the solution should be. The primary optimization goal is to minimize the number of containers used N. In case two solutions use the same number of containers, the one that utilizes the space more efficiently—i.e., with a higher minimum fill ratio (volume or weight) is considered better. Combining these two requirements gives us the following formula:

let $\mathcal{U}$ be the set of used (i.e., opened and non-empty) containers,

then the fitness function is defined as:

$$\text{fitness} = N + \min_{c \in \mathcal{U}} \left(\max \left(\frac{V(c)}{V(D)}, \frac{W_c}{W_{\max}} \right) \right).$$

TODO: nejnizsi odhad TODO: kdy neni solution

1.8 Data Representation

1.8.1 Input

The input consists of a description of the input container and a list of input boxes. The container is defined by its dimensions $D = (X, Y, Z)$ and its maximum weight $W_{\max}$. Each input box i is described by a unique identifier, its dimensions $d_i = (x, y, z)$, and its weight w_i. The input is provided in JSON format, where all numerical values are expressed in the same units. Example inputs can be found here:

`https://gitlab.mff.cuni.cz/teaching/nprg045/fink/jan_burda/-/tree/master/Code/TestInputs?ref_type=heads`.

1.8.2 Output

The output is a JSON object containing an array of containers.

Each container includes:

- its unique identifier,
- total weight,
- total occupied volume,
- a list of packed boxes.

Each box entry includes:

- the box identifier,
- applied rotation,
- the region it occupies.

All numerical values are expressed in the same unit system as the input. An example output can be found here:

`https://gitlab.mff.cuni.cz/teaching/nprg045/fink/jan_burda/-/blob/master/Code/Documentation/OutputExample.json?ref_type=heads`.

1.8.3 Datasets

Part of the datasets was generated randomly. The method consists of creating random input boxes within predefined sizes limits, and assigning their weights based on a randomly generated density, ensuring a consistent relationship between volume and weight.

2 The Packing Algorithm

This chapter describes how we can obtain a deterministic packing solution, i.e. a list of containers and boxes packed within them, from the packing input and the packing rules. The packing rules contain information regarding the rotation of each box, the heuristic that will determine how the box is placed in the container, and the order in which the boxes are packed.

First, the list of input boxes is converted into an ordered list of pending boxes using the packing rules. Each pending box contains the input box together with the information about its rotation and the heuristic that will be used to pack the box.

Once we have the list of pending boxes, they can be packed using the box packing algorithm that takes each pending box and tries to pack it into an open container chosen by the placement heuristic assigned to that particular pending box. In case no fitting container and space are found, a new container is opened. Once a pending box is placed into a container, we call it a placed box. This placed box consists of the input box information, its rotation, the space it occupies within the container, and the identifier of that container.

The solution is then a list of open containers, each with a list of placed boxes within that particular container. Since we can assign fitness to each solution, and the packing rules and packing input determine a unique solution, we can also assign fitness to the packing rules themselves with regard to the packing input.

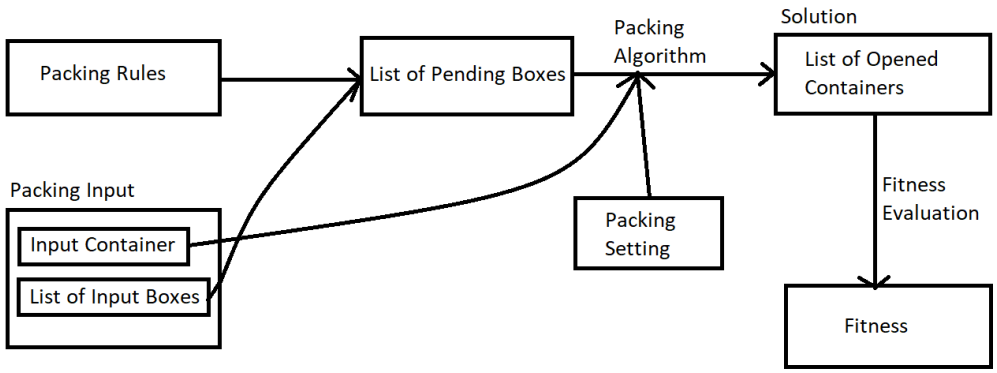

Figure 2.1

2.1 Terminology

2.1.1 Pending box

A *pending box* is represented as a triple (B, Rot, H), where:

- B is the input box,
- Rot is the selected rotation,
- H is the placement heuristic.

The reason each pending box has its own placement heuristic, rather than using a single heuristic for all pending boxes, is to support variability in evolutionary algorithms.

2.1.2 Placement Heuristic

A *placement heuristic* is a function that takes a list of currently opened containers and an input box with its rotation, and returns a selected container along with the region where the box should be placed. Since the input is defined only as a list of opened containers, the heuristic has considerable flexibility. It can take into account various factors when choosing the target container and placement space. For example, if the heuristic had access only to the EMR lists of containers, its decision-making would be restricted to available empty regions. In this more general formulation, however, it can also consider additional attributes such as the weight or volume of each container.

If no valid placement for the box is found (i.e., there is no EMR in any container with sufficient dimensions, or placing the box would exceed the container's maximum weight), then the function returns *null*.

2.1.3 Opened Container

An *opened container* C is represented as a triple (C_{in}, B_c, EMR_c), where:

- C_{in} represents the input container,
- B_c is the set of boxes already placed into the container,
- EMR_c is the set of Empty Maximal Regions within the container.

2.1.4 Placed Box

A *placed box* is represented as (B, C, Reg, Rot), where:

- B is the box,
- C is the container into which the box is placed,
- Reg is the region occupied by the box within the container,
- Rot is the rotation of the box.

2.1.5 Packing Rules

Let $\mathcal{H}$ denote the set of all available placement heuristics, and let $\mathcal{R}$ denote the set of all allowed box rotations. The *packing rules* provide a compact representation of the list of pending boxes and define the order in which they are packed. Formally, they are represented as a triple (o, rot, h), where:

- o is a permutation list of integers from 1 to k. The value at index i specifies the packing order of the box with identifier i.

- rot is a list of length k, where the element at index i represents the rotation assigned to the input box with identifier i, $rot \in \mathcal{R}^k$.
- h is a list of length k, where the element at index i represents the placement heuristic assigned to the input box with identifier i, $h \in \mathcal{H}^k$.

In other words, the packing rules determine the packing order, rotation, and placement heuristic for each input box, which together form the corresponding list of pending boxes.

Vector representation

In order to use the packing rules as an individual for the evolutionary algorithm, they can be represented as a continuous vector of length $3k$ consisting of real values in the interval $[0, 1]$. [5] This vector is divided into three parts corresponding to the components (o, rot, h).

In this triple, the first k entries are sorted in ascending order to derive the permutation o, the second k entries are interpreted as uniformly distributed indices to select elements from the set of allowed rotations $\mathcal{R}$, and the third k entries are interpreted similarly to choose elements from the set of available heuristics $\mathcal{H}$. This transformation allows evolutionary algorithms to operate on the vector representation while still producing valid discrete packing rules.

If the set $\mathcal{H}$ contains only a single heuristic, the heuristic part becomes redundant, and the vector can be reduced to two parts with a total length of $2k$. A similar situation arises if rotations are not allowed, meaning that no rotation can be applied and the only feasible orientation is the default XYZ. In this case, the rotation part is unnecessary, and the vector can also be shortened to length $2k$. If neither of these parts are needed, the vector can be shortened further to length k. Furthermore, if the packing order is determined directly by a heuristic, the order part can also be omitted, as the order can be derived without relying on the packing vector.

2.1.6 Packing Rules Fitness

Previously, the concept of fitness was defined in the context of evaluating a solution. It is now evident that the packing input together with packing rules uniquely determine a specific solution. This allows fitness to be assigned not only to the solution itself but also to the packing rules with respect to the given packing input. The procedure is as follows: the packing rules are first converted into a list of pending boxes in a specified order, the packing algorithm is then applied to generate the solution, which is subsequently evaluated to determine its fitness.

2.2 Empty Maximal Regions

To keep track of unoccupied regions during the packing process, we assign each opened container a list of *empty maximal regions* (EMR). An empty region is said to be *maximal* if it is not a subregion of any other empty region. In the literature, the more common term is empty maximal spaces (EMS). However, the

term regions is adopted here to ensure consistency with the terminology of this work. [6, 5]

Initially, each empty opened container c has a single EMR, which is the region of the container itself, denoted as R_c. As boxes are placed into the container, the set of EMR is updated accordingly.

2.2.1 Update of a Single Empty Maximal Region

Assume we have an Empty Maximal Region:

$$\text{EMR} = (\text{Start}, \text{End}) = ((x_s, y_s, z_s), (x_e, y_e, z_e)),$$

and a region now occupied by a newly placed box:

$$\text{BoxRegion} = ((x_b^s, y_b^s, z_b^s), (x_b^e, y_b^e, z_b^e)).$$

If EMR and BoxRegion do not intersect, the original EMR remains unchanged. If they do intersect, we define their intersection as:

$$\text{IntersectRegion} = ((x_i^s, y_i^s, z_i^s), (x_i^e, y_i^e, z_i^e)) = \mathit{EMR} \cap \mathit{BoxRegion}.$$

In the general case, this intersection splits the original EMR into up to six new EMR subregions, as follows:

- **Left slab (along X)**:

$$((x_s, y_s, z_s), (x_i^s, y_e, z_e))$$

- **Right slab (along X)**:

$$((x_i^e, y_s, z_s), (x_e, y_e, z_e))$$

- **Front slab (along Y)**:

$$((x_s, y_s, z_s), (x_e, y_i^s, z_e))$$

- **Back slab (along Y)**:

$$((x_s, y_i^e, z_s), (x_e, y_e, z_e))$$

- **Bottom slab (along Z)**:

$$((x_s, y_s, z_s), (x_e, y_e, z_i^s))$$

- **Top slab (along Z)**:

$$((x_s, y_s, z_i^e), (x_e, y_e, z_e))$$

Each of these regions is only created if it has positive volume and satisfies the definition of a valid region, i.e., its start coordinates are less than or equal to its end coordinates in all dimensions.

When constraint C6 is applied, we have $z_b^s = z_s$, and thus the bottom slab is never created, reducing the number of new EMR to at most five. Another consequence of constraint C6 is that the top slab must lie directly above the placed box. Therefore, its x and y coordinates are restricted to match the footprint of the box, making it more compact than in the general case:

- **Top slab (along Z)**:

$$((x_i^s, y_i^s, z_i^e), (x_i^e, y_i^e, z_e)).$$

It is worth explicitly mentioning that there is no requirement for the newly generated EMR to be non-overlapping. Overlaps are allowed and common in this representation.

`https://www.geogebra.org/calculator/zatgrfrg`

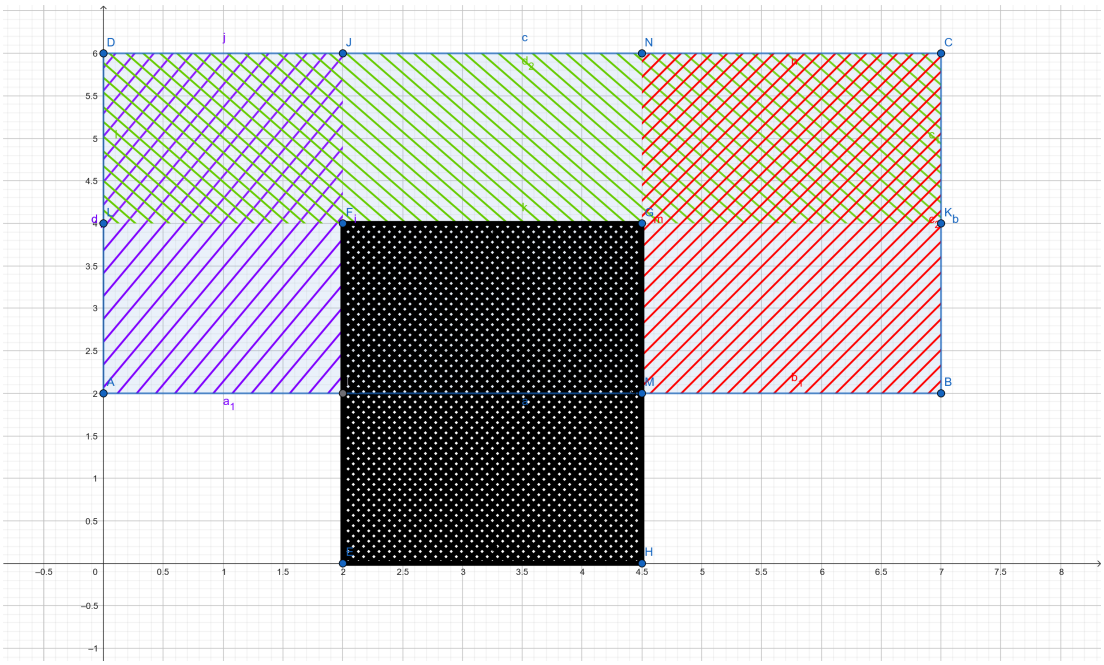

Figure 2.2 A simplified 2D example of EMR update. Suppose we have an EMR defined as (A, C), and a newly placed box occupying the region (E, G). The intersection region is (I, G), and three new EMR are created: (A, J), (L, C), and (M, C).

2.2.2 Update of an Empty Maximal Regions List

Since a newly placed box can intersect multiple EMR, it is necessary to update each region in the EMR list using the method described above. Once this is done, several additional steps must be taken.[5]

First, we check whether any of the newly created EMR is a subregion of another EMR in the list. Such subregions do not satisfy the definition of maximality and are therefore removed. Second, we attempt to merge adjacent EMR that lie next to each other and start at the same height. Due to constraint C6, each newly

generated top slab lies directly above the placed box. However, the constraint still allows a box to rest on top of multiple smaller boxes, as long as its base is fully supported. This means that EMR starting at the same height and sharing a face can potentially be merged into a larger EMR.

2.3 Heuristics

2.3.1 Packing Order Heuristics

When using evolutionary algorithms, finding a sufficiently good solution often requires considerable time. An alternative approach is to employ heuristics to determine the order of the pending boxes a priori. Some ordering heuristics can produce high quality solutions almost instantaneously. However, they tend to converge quickly and, in the long term, are generally outperformed by the evolutionary approach to determining the packing order.

The most effective results were obtained using a heuristic that sorts the pending boxes by their volume, placing the largest boxes first. Other similar strategies include sorting by base area, by the longest side, or by weight in cases where containers are more frequently constrained by weight than by volume.

2.3.2 Placement Heuristics

Several placement heuristics can be employed to choose a suitable EMR in which a box will be placed. The following list summarizes the main approaches.

1. **Best Fit**
   Selects the EMR with the smallest volume that can still accommodate the box. By choosing the EMR with minimal remaining free space, this heuristic aims to reduce fragmentation of unused volume and promote compact packings.

2. **Back–Bottom–Left (BBL)**
   A classical rule that positions boxes as far as possible toward the back, then the bottom, and finally the left side of the container. The placement is therefore determined sequentially: the heuristic first minimizes the X coordinate, then Z and lastly Y.

3. **DFTRC (Distance to the Front–Top–Right Corner)**
   For each EMR capable of accommodating the box, compute the distance from the front–top–right corner of the container to the minimal coordinate of the EMR. The EMR that maximizes this distance is selected, as a greater DFTRC implies a more compact packing.

2.4 The Box Packing Algorithm

Given a list of ordered pending boxes, ordered according to a predefined packing sequence, the algorithm iteratively places each box into one of the currently opened containers. If none of the existing containers can accommodate the box, a new container is opened. The process continues until all boxes are packed.

Pseudocode

```
Input: List of ordered pending boxes B = [b_1, b_2, ..., b_n]
Output: List of opened containers C = [c_1, c_2, ..., c_m]
Initialize empty list of opened containers C ← []
for each (B, Rot, H) in B do
    (C, Reg) ← H(C, (B, Rot))                    ▷ Apply placement heuristic
    if (C, Reg) = null then
        Open a new container C_new
        Add C_new to C
        (C, Reg) ← H(C, (B, Rot))
        if (C, Reg) = null then
            Error: Unable to place box in any container
        end if
    end if
    Add placed box (B, C, Reg, Rot) to the set of boxes placed in C
    Update the EMR list of container C based on placement Reg
end for
return C
```

3 Packing Rules Evolution

This chapter introduces evolutionary algorithms as a method to improve the quality of packing solutions. The main idea is to evolve a population of candidate solutions, called individuals, over a number of iterations, using principles inspired by natural evolution.

The evolutionary process evaluates the fitness of each individual, selects the more promising solutions according to their fitness, and applies variation operators to generate new candidate solutions. Through repeated iterations, the population is gradually refined, searching for high-quality individuals that lead to more efficient solutions.

In the context of the packing problem, each individual represents the packing rules in its vector form, defining a distinct way to generate a deterministic packing solution.

3.1 Hill Climbing

Hill Climbing is the simplest of the evolutionary algorithms. For each individual, a child is generated by applying small mutations according to a predefined mutation probability. After evaluating the child's fitness, either the parent or the child is selected for the next iteration based on which has higher fitness.

This approach works because each individual explores a small neighborhood in the solution space, but since the population contains many individuals, the overall search covers a much larger region. In other words, while each hill climbing step is local, the collective exploration by the population allows the algorithm to find high-quality individuals efficiently.

In this work, mutation is applied by independently replacing each element of the packing-rules vector with a random value sampled uniformly from the interval [0,1] according to a predefined mutation probability. This alters box order, rotations, or heuristic preferences, producing a modified candidate solution that is evaluated against its parent.

- **Mutation probability** — the probability that element in the vector will be replaced with a random value in the interval [0,1].
- **Population size** — the number of individuals that undergo independent Hill Climbing.
- **Number of generations** — total number of Hill Climbing iterations applied to each individual.

3.2 Simulated Annealing

Simulated Annealing is similar to Hill Climbing in that it iteratively generates a new candidate solution from a current solution, typically by applying small mutations to the individual's vector representation. The key difference is that SA can accept worse solutions with a probability that depends on a temperature parameter, which gradually decreases over time.

The temperature controls the likelihood that a child solution will replace its parent even if it has lower fitness. Early in the search, when the temperature is high, worse solutions are more likely to be accepted, promoting exploration of the solution space. As the temperature decreases, the acceptance probability of worse solutions declines, making the algorithm behave more like Hill Climbing and focus on local improvements.

The temperature is decreased each generation according to a geometric cooling schedule:

$$T_{next} = \alpha \cdot T_{current},$$

where $\alpha \in (0, 1)$ is the cooling rate. Early iterations allow greater exploration, while later iterations focus on local refinement.

- **Initial temperature** — the starting value of the temperature controlling the acceptance probability of worse solutions.
- **Cooling rate** — the factor by which the temperature decreases each iteration.
- **Mutation probability** — the probability that an element in the vector will be replaced with a random value in the interval [0,1].
- **Population size** — the number of individuals that undergo independent Simulated Annealing.
- **Number of generations** — total number of iterations applied to each individual.

3.3 Elitist Genetic Algorithm

The Elitist Genetic Algorithm evolves a population of individuals by combining and modifying existing solutions while giving special treatment to the best-performing individuals. [5]

A subset of the population, called the *elite*, consists of the individuals with the highest fitness. Elite solutions are preserved across generations and updated after each generation to ensure that the best traits are not lost.

New individuals are generated using a combination of selection, crossover, and mutation. One parent is selected from the elite subset, while the other is chosen from the remainder of the population. For each element of the offspring vector, a value is inherited from one of the parents according to a predefined probability, typically giving a higher chance to inherit from the elite parent. This allows beneficial traits from the elite to propagate while maintaining diversity.

After crossover, mutation is applied. This introduces additional variation, potentially altering box order, rotations, or heuristic preferences.

Additionally, a small number of entirely random solutions are introduced into the population each generation. These individuals further increase diversity and help prevent premature convergence to local optima.

The newly generated solutions replace part of the non-elite population, while the elite remain unchanged.

In this work, mutation is applied by independently replacing each element of the packing-rules vector with a random value sampled uniformly from the interval [0,1], according to the mutation probability parameter.

For each crossover, one parent is selected from the elite subset using tournament selection with the tournament size given by a parameter, and the other parent is selected randomly from the non-elite subset

- **Population size** — total number of candidate solutions in the population.
- **Elite size** — number of top-performing individuals preserved each generation.
- **Mutation probability** — probability that an element in the packing-rules vector will be replaced with a random value.
- **Crossover bias** — probability of inheriting an element from the elite parent during uniform crossover.
- **Tournament size** — number of elite candidates considered when selecting a parent via tournament selection.
- **Number of generations** — total number of iterations the algorithm runs.
- **Number of random individuals** — number of entirely new solutions introduced into the population each generation.

3.4 Memetic Extension of the Elitist Genetic Algorithm

The Memetic Elitist Genetic Algorithm extends the Elitist Genetic Algorithm by applying a local search to newly generated packing rules. After crossover and mutation, each offspring undergoes a short Hill Climbing procedure, refining the solution in its local neighborhood. This combines the global search capability of the genetic algorithm with the ability of local search, often resulting in faster convergence and higher-quality solutions.

The extent of the local search is controlled by a parameter specifying the number of Hill Climbing iterations applied to each offspring. In this implementation, Hill Climbing is executed for the given number of iterations, producing several candidate solutions. The best among these candidates is selected. If none of the candidates improves over the original offspring, the original solution is retained. This allows balancing between computational cost and solution refinement. A higher number of iterations may improve the solution quality but increases runtime, while a smaller number keeps the process faster but explores less of the local neighborhood.

- **Number of Hill Climbing iterations** — number of independent Hill Climbing runs performed per offspring; the best result is chosen or the original solution is kept if none is better.
- **Hill Climbing mutation probability** — probability that an element in the packing-rules vector is mutated during the memetic Hill Climbing step.

Bibliography

1. Zizek, Slavoj. *Multiculturalism, or, the cultural logic of multinational capitalism*. New Left Review, 1997.
2. Bernhofen, Daniel M.; El-Sahli, Zouheir; Kneller, Richard. Estimating the effects of the container revolution on world trade. *Journal of International Economics*. 2016, vol. 98, pp. 36–50. issn 0022-1996. Available from doi: https://doi.org/10.1016/j.jinteco.2015.09.001.
3. OByrne, Rob. *Shipping container utilisation: The challenge and why it matters*. 2021. Available also from: https://www.benchmarkingsuccess.com/the-challenge-of-freight-container-utilisation-and-why-it-matters/.
4. Martello, Silvano; Pisinger, David; Vigo, Daniele. The Three-Dimensional Bin Packing Problem. *Operations Research*. 2000, vol. 48, no. 2, pp. 256–267. Available from doi: 10.1287/opre.48.2.256.12386.
5. Gonçalves, José Fernando; Resende, Mauricio G.C. A biased random key genetic algorithm for 2D and 3D bin packing problems. *International Journal of Production Economics*. 2013, vol. 145, no. 2, pp. 500–510. issn 0925-5273. Available from doi: https://doi.org/10.1016/j.ijpe.2013.04.019.
6. Lai, K.K.; Chan, Jimmy W.M. Developing a simulated annealing algorithm for the cutting stock problem. *Computers and Industrial Engineering*. 1997, vol. 32, no. 1, pp. 115–127. issn 0360-8352. Available from doi: https://doi.org/10.1016/S0360-8352(96)00205-7.